

SPATIAL STAY-POINT DETECTION

Con k-means e DBSCAN

Davide Bianchi, Mattia Gallotta

10 novembre 2023

Sommario

Lo studio di dati geospaziali, noto anche come GIS (Geographic Information System), è un campo specializzato nell'ambito dell'informatica e della geografia che si occupa della raccolta, dell'analisi e della visualizzazione di dati geografici e spaziali. Una branca di questa macro-area è dedicata alla rilevazione degli “stay point” ovvero dei “punti di permanenza”. Il presente progetto si pone come obiettivo di applicare le conoscenze apprese durante il corso di GPU Computing per parallelizzare due algoritmi di clustering: k -means e DBSCAN, adoperati nell'analisi di tali dati. Il caso preso in esame è la visita di un museo a Trento dove sono stati studiati i punti di permanenza dei visitatori per poi associarli ai punti di interesse della struttura.

Indice

1	Introduzione	2
1.1	Stay point detection	2
1.2	Dataset	2
1.3	K-means	2
1.4	DBSCAN	3
2	Strutture dati	3
2.1	Strutture comuni	3
2.2	Strutture Parallel k-means	3
2.3	Strutture G-DBSCAN	3
3	Implementazione seriale (CPU)	4
3.1	Parallel k-means	4
3.2	G-DBSCAN	5
4	Implementazione parallela (GPU)	6
4.1	Parallel k-means	6
4.2	G-DBSCAN	6
5	Speedup e profiling	8
5.1	Parallel k-means	8
5.2	G-DBSCAN	9
6	Valutazione degli stay point	10
7	Conclusione	12

1 Introduzione

1.1 Stay point detection

La stay point detection è una tecnica fondamentale nell'analisi dei dati di mobilità utilizzata per identificare i punti di permanenza a partire da un insieme di traiettorie o sequenze di posizioni geospaziali. Questo risulta particolarmente rilevante poiché, durante il movimento, un individuo o un oggetto possono fermarsi in punti specifici. Questi, noti come stay points, sono spesso luoghi di interesse come abitazioni, siti di lavoro, negozi, parchi o simili.

1.2 Dataset

I dati presi in esame sono coordinate geospaziali bidimensionali (x,y) che rappresentano i punti di movimento dei visitatori all'interno di un museo. Il dataset, salvato in un file csv, si compone di 32.371 campioni raccolti con una frequenza di 12 Hz per una distanza complessiva di 1.787 metri. Per avere un'idea più chiara ed intuitiva, si propone l'immagine 1.

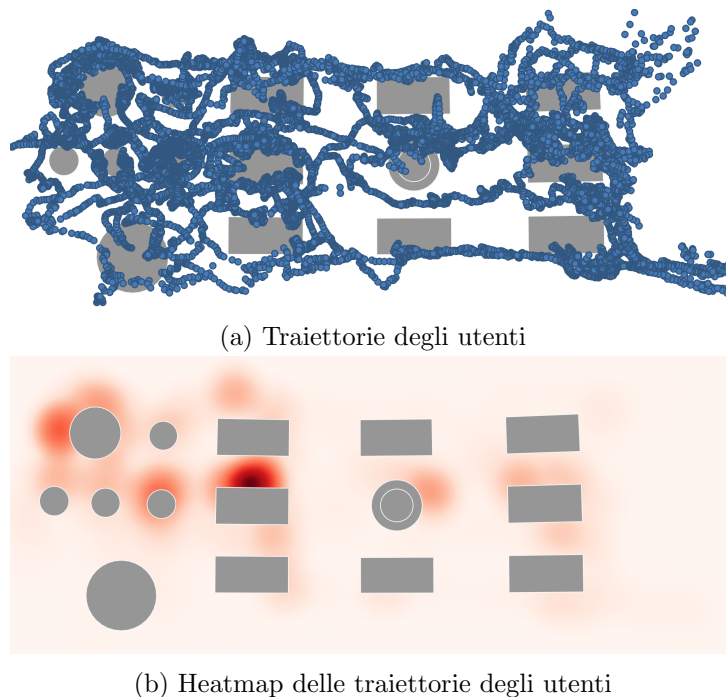


Figura 1: Presentazione dei punti del dataset

1.3 K-means

Il primo approccio studiato per l'analisi dei dati è l'algoritmo di clustering k -means, uno tra i più noti attualmente utilizzati. Questo prende in input un dataset di n oggetti (in questo caso punti nello spazio bidimensionale) e li raggruppa in k partizioni chiamati cluster sfruttando un criterio di similarità basato sulla distanza. L'algoritmo si articola come segue:

1. Seleziona k oggetti dal dataset che rappresentano i centri iniziali dei cluster;
2. Assegna ogni oggetto rimanente nel dataset ad uno dei k cluster misurando la distanza tra il punto preso in esame e i k centri. Un punto viene assegnato al cluster più vicino;
3. Calcola il punto medio di ogni cluster che sarà utilizzato come nuovo centro.

Il processo viene ripetuto finché un determinato criterio viene soddisfatto come ad esempio il numero massimo di iterazioni o la convergenza dei centroidi.

1.4 DBSCAN

Il secondo metodo utilizzato è l'algoritmo di clustering DBSCAN. Esso si differenzia da k -means perché la clusterizzazione dei punti è basata sulla densità dei dati e non sulla distanza. Questo comporta la creazione di cluster dalle varie forme e non solamente circolari, inoltre, il numero di cluster non deve essere noto in anticipo come per k -means. Solitamente, per definizione, questa soluzione è preferibile per rilevare gli stay point. L'algoritmo di articola come segue:

1. Partendo dal dataset, costruisce un grafo dove ogni nodo rappresenta un oggetto e ogni arco la connessione tra di essi. Due campioni sono connessi quando la loro distanza è minore o uguale rispetto un raggio R definito come parametro;
2. Classifica un nodo come *Core* se il suo grado, ovvero il numero di nodi vicini, è superiore ad un parametro *min_pts* specificato;
3. Esplora in ampiezza il grafo creato allo step precedente partendo dai nodi *Core*. A seguito di questa fase, i nodi esplorati vengono definiti *Border* (i.e., nodi periferici), mentre quelli non raggiungibili sono definiti come *Noise*;
4. Identifica come cluster ogni insieme di oggetti associati ad un *Core*.

2 Strutture dati

In questa sezione presentiamo le strutture dati utilizzate dagli algoritmi per il clustering. Si noti che tutte sono state implementate con delle Struct of Array (SoA) per questioni di coalescenza.

2.1 Strutture comuni

Per salvare le informazioni riguardanti gli utenti abbiamo definito la struttura `Samples` formata dai seguenti campi:

- `user`: un array di stringhe per memorizzare il codice degli utenti;
- `x` e `y`: due array di `float` per le coordinate spaziali bidimensionali.

2.2 Strutture Parallel k-means

- `KeyValueMap`: la struttura usata per salvare il cluster di appartenenza di ogni sample. Essa è composta dai seguenti campi:
 - `key`: un array di `int` che memorizza gli indici dei cluster ai quali i punti sono assegnati;
 - `x` e `y`: due array di `float` per le coordinate spaziali degli oggetti del dataset;
 - `size`: un `int` per tenere traccia della dimensione corrente della mappa.
- `Centers`: la struttura che memorizza le coordinate bidimensionali dei centri attraverso i due array `x` e `y` di `float`.

2.3 Strutture G-DBSCAN

Le seguenti strutture fanno uso di 2 nuovi tipi: `node_sz` e `node` entrambi `unsigned int`. Di seguito la loro descrizione:

- `GraphStruct`: usata per la gestione del grafo (identica a quella del laboratorio del corso). Essa è composta dai seguenti campi:
 - `node_size` ed `edge_size`: due variabili di tipo `node_sz` usate per memorizzare rispettivamente il numero di nodi e di archi del grafo;

- `cumDeps`: un array di tipo `node_sz` usato per salvare la lista dei gradi cumulati;
- `neighs`: un array di tipo `node` per la lista di adiacenza dei nodi.
- `Nodes`: usata per l'esplorazione del grafo. Essa è composta dai seguenti campi:
 - `visited`: un array di tipo `int` per marcare i nodi già visitati;
 - `type`: un array di tipo `Type` per salvare la tipologia del nodo (i.e., Core, Border e Noise);
 - `cluster`: un array di `int` per indicare il cluster di appartenenza di ogni nodo.

3 Implementazione seriale (CPU)

3.1 Parallel k-means

La soluzione proposta da [1] si basa su un modello di programmazione Map-Reduce assumendo che i dati siano distribuiti su più computer ed è composto da tre funzioni principali:

- `Map`: assegna ogni sample al centro più vicino. Prende in input n coppie chiave-valore che rappresentano i record del dataset; più precisamente, la chiave rappresenta il numero progressivo del record e il valore la posizione del punto nello spazio. In output sono prodotte delle coppie chiave-valore la cui chiave è l'indice del cluster rappresentato dal centro più vicino, mentre il valore rimane invariato.
- `Combine`: combina i risultati delle Map sul singolo calcolatore. Prende in input l'indice `key` del cluster e tutti i sample ad esso assegnati e produce in output delle coppie chiave-valore la cui chiave è l'indice del cluster e il valore è una coppia formata dal numero di elementi del cluster e la loro somma.
- `Reduce`: calcola i nuovi centri dei cluster. Prende in input l'indice `key` del cluster e la lista delle somme parziali fatte da ogni host e produce delle coppie chiave-valore che rappresentano gli indici dei cluster e i nuovi centri.

A seguito di un'attenta analisi e in base alla disponibilità delle risorse, si è preferito modificare l'algoritmo per meglio adattarlo ad un singolo host. L'operazione di Map è rimasta invariata, mentre le operazioni Combine e Reduce sono state unite in un'unica funzione. Inoltre, per poter confrontare la soluzione seriale e quella parallela, si è utilizzata l'inizializzazione di k -means ++ come mostrata in [2].

L'implementazione ha visto lo sviluppo delle seguenti funzioni:

- `assign_cluster`: è il corrispettivo della funzione Map. Essa calcola la distanza tra il punto in esame e ogni centroide tramite la funzione `calculate_distance` e gli associa l'indice del cluster più vicino. Questo record viene aggiunto alla mappa;
- `explore_samples`: un wrapper della funzione `assign_cluster` che consente di eseguirla per ogni sample del dataset;
- `calculate_new_centroids`: calcola il punto medio (di coordinate x e y) di tutti i nodi associati ad un cluster. Il risultato sarà utilizzato come nuovo centro;
- `update_centroids`: un wrapper della funzione `calculate_new_centroids` per calcolare i nuovi centri di tutti i cluster in esame;
- `check_threshold`: controlla per tutti i nuovi centri calcolati se discostano almeno di una soglia E . In caso affermativo, l'algoritmo viene eseguito nuovamente, altrimenti termina dato che i centroidi convergono.

Per evidenziare come queste componenti interagiscono, si propone lo pseudo-codice della funzione principale mostrato nell'algoritmo 1.

Algoritmo 1 Funzione principale del Parallel k-means CPU

```

1: procedure CPU_PKMEANS(dataset, centers)
2:   map ← malloc_map()
3:   read_sample(dataset)
4:   stop_condition ← false
5:   cont ← 0
6:   clusters ← array di lunghezza N_SAMPLES
7:   while not stop_condition do
8:     init_map(map)
9:     explore_samples(dataset, centers, map, clusters)
10:    new_centers ← malloc_centers()
11:    copy_centers(new_centers, centers)
12:    update_centroids(map, new_centers)
13:    stop_condition ← check_threshold(centers, new_centers)
14:    centers ← new_centers
15:    cont ← cont +1
16:   end while
17:   return clusters
18: end procedure

```

3.2 G-DBSCAN

La soluzione proposta da [3] può essere diviso in tre funzioni principali:

- **MakeGraph**: Essa prende in input: il dataset, R , min_pts , e inizializza il grafo. Più precisamente, essa costruisce un grafo $G(V,E)$ dove V rappresenta l'insieme dei vertici, ovvero gli oggetti da clusterizzare, ed E rappresenta l'insieme degli archi che connettono gli oggetti che si trovano ad una distanza minore del raggio di prossimità R . Le operazioni sono state implementate dalla funzione `make_graph` che a sua volta utilizza `calculate_distance` per calcolare la distanza euclidea tra i punti e `classify_object` per individuare i punti *Core*;
- **IdentifyCluster**: dopo la creazione del grafo e la classificazione degli oggetti, il prossimo step consiste nell'identificazione dei cluster. Per fare ciò, questa funzione effettua un'esplorazione in ampiezza partendo da un nodo classificato come *Core* e visitando tutti gli altri vertici raggiungibili da esso. Man mano che i vertici sono esplorati, vengono classificati come *Border* e sono etichettati come membri dello stesso cluster. Il processo di visita viene ripetuto su tutti i vertici *Core* che non sono ancora stati visitati precedentemente. Le operazioni sono state implementate da `identify_cluster` che a sua volta chiama la funzione `breadth_first_search`.
- **BFS**: esplora in ampiezza il grafo partendo da un nodo. Prende in input la sorgente, il grafo e il numero di cluster. Questo metodo si basa su due maschere di appoggio:
 - La frontiera F_a : marca quali nodi devono essere esplorati nella iterazione corrente;
 - La lista dei nodi esplorati X_a : indica quali nodi sono stati visitati.

L'esplorazione viene eseguita per livelli, sulla frontiera si troverà inizialmente il nodo sorgente e ad ogni iterazione sono raggiunti i nodi del grafo ad una distanza crescente da esso. Per raggiungere questo obiettivo, si sono sviluppate le funzioni `breadth_first_search` e

`breadth_first_search_kernel`. La prima si occupa di effettuare la visita per livelli e assegnare il numero di cluster a tutti i nodi esplorati, mentre la seconda si occupa di visitare un singolo livello.

Per evidenziare come queste componenti interagiscono, si propone lo pseudo-codice della funzione principale mostrato nell'algoritmo 2.

Algoritmo 2 Funzione principale di CPU DBSCAN

```

1: procedure CPU_DBSCAN(dataset)
2:   nodes ← init_nodes()
3:   graph ← create_empty_graph()
4:   initialize_graph(graph, dataset)
5:   make_graph(MIN_PTS, R, dataset, graph, nodes)
6:   identify_cluster(graph, nodes)
7:   return nodes.cluster
8: end procedure

```

4 Implementazione parallela (GPU)

4.1 Parallel k-means

Sulla base della descrizione dell'algoritmo precedentemente presentato, si può notare che il calcolo della distanza tra un sample del dataset e un centroide è indipendente e quindi può essere parallelizzato. Per questo motivo, la funzione Map può essere eseguita agevolmente da tanti thread pari al numero n di oggetti; la funzione Reduce, invece, può essere eseguita da esattamente k thread pari al numero dei cluster.

L'implementazione ha visto lo sviluppo dei seguenti kernel:

- `d_assign_cluster`: rappresenta la controparte parallela della versione CPU. I calcoli sono i medesimi, ma eseguiti da un numero di thread pari alla dimensione del dataset. Ad ognuno è stato assegnato un identificativo lineare globale per accedere agli oggetti da confrontare con ogni centro;
- `d_update_centroids`: rappresenta la controparte parallela della versione CPU. I calcoli sono i medesimi, ma eseguiti da un numero di thread pari al numero di cluster k . Ad ognuno è stato assegnato un identificativo lineare globale per accedere agli oggetti appartenenti ad ogni cluster e calcolare il punto medio. Al suo interno viene utilizzata la controparte GPU della funzione per il calcolo dei nuovi centroidi chiamata `d_calculate_new_centeroids`.

La funzione principale per il clustering GPU è analoga a quella CPU mostrata nell'algoritmo 1. Le principali differenze riguardano i trasferimenti di dati tra host e device e le chiamate dei kernel.

4.2 G-DBSCAN

Sulla base della descrizione dell'algoritmo precedentemente presentato, si può notare che possono essere parallelizzati sia la costruzione che l'esplorazione del grafo.

L'implementazione della `MakeGraph` ha visto lo sviluppo dei seguenti kernel:

- `calculate_degs`: calcola i gradi di ogni nodo. Prende in input il dataset, l'array dei gradi di output, il contatore degli archi, il grafo e il raggio. Al termine dell'esecuzione i gradi di ogni nodo e il numero di archi totali saranno salvati nelle rispettive variabili. Il kernel viene lanciato su una griglia di thread bidimensionale dai quali sono stati estratti gli indici

di riga i e di colonna j . Grazie ad essi è possibile accedere agli oggetti del dataset a coppie, calcolare la distanza tra di essi e vedere se sono connessi o meno. Se i due nodi sono connessi il numero di archi e il grado ad esso associati sono aumentati di uno.

- **naive_scan**: partendo dalla lista dei gradi calcolati allo step precedente, esso crea la lista dei gradi cumulati a livello di blocco. L'esecuzione adopera la shared memory per poter velocizzare le letture. Si utilizzano due tipi di indicizzazione: lineare globale per la lettura e scrittura dei dati in memoria global e lineare locale per le elaborazioni in memoria shared. La dimensione della memoria shared è pari al doppio di quella del blocco. Inizialmente, ogni thread ha il compito di caricare un dato dalla lista dei gradi in memoria global nella prima metà della memoria shared. Una volta sincronizzati, iterativamente, ogni thread effettua le somme intermedie tra elementi a distanza **offset** pari a 1 che saranno memorizzate nella seconda metà della memoria. Nelle iterazioni successive, la shared viene utilizzata in maniera alternata per raggiungere lo stesso obiettivo del passo precedente, con **offset** che raddoppia ad ogni step fino alla dimensione del blocco. Per visualizzare questo funzionamento, si rimanda alla figura 2.
- **add_block_sums**: questo kernel si occupa della creazione della lista dei gradi cumulati completa partendo dalla lista dei gradi cumulata a livello di blocco. Si prenda in esempio il blocco di thread 1, questo somma agli elementi del blocco 1, il valore dell'ultimo elemento del blocco 0; questo compito viene ripartito tra i thread. Analogamente, il blocco 2 somma il valore dell'ultimo elemento del blocco 1, e così via fino al blocco n dove agli elementi viene sommato il valore dell'ultimo elemento del blocco $n - 1$.
- **prepend_zero**: antepone il valore 0 alla lista dei gradi cumulati. Ogni thread ha il compito di spostare a destra un elemento dell'array ad eccezione del primo thread che scrive uno zero in posizione 0.
- **make_adj_list**: crea la lista di adiacenza. Ogni thread confronta un nodo con tutti gli altri e, se la distanza tra i due è minore del raggio R , salva il valore dell'altro nodo nella lista di adiacenza partendo dalla sua sezione di competenza.
- **d_classify_objects**: classifica i nodi del grafo come *Core* se il numero di vicini supera il parametro *min_pts*.

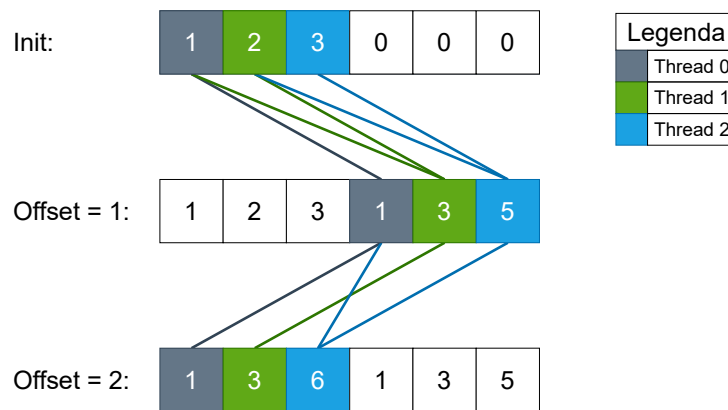


Figura 2: Esecuzione della naive scan parallela di [1,2,3]

L'implementazione della BFS ha visto lo sviluppo dei seguenti kernel:

- **breadth_first_search_gpu_kernel**: si occupa di esplorare in ampiezza i nodi presenti nella frontiera a partire da una sorgente comune. Prende in input la lista di adiacenza e dei

gradi cumulati, la frontiera F_a e la lista dei nodi visitati X_a . Al termine dell'esecuzione, la maschera X_a sarà aggiornata con i vertici raggiunti nella fase di esplorazione.

- **assign_cluster**: ha come compito di assegnare ad ogni nodo nella lista degli esplorati il numero di cluster passato come parametro. Prende in input il grafo e la struttura di supporto `Nodes`. Al termine dell'esecuzione, ad ogni nodo della lista degli esplorati sarà assegnato il proprio numero di cluster.

La funzione principale per il clustering GPU è analoga a quella CPU mostrata nell'algoritmo 2. Le principali differenze riguardano i trasferimenti di dati tra host e device e le chiamate dei kernel.

5 Speedup e profiling

I test sono stati effettuati su Google Colab, utilizzando una GPU NVIDIA Tesla T4 con 15 GB di RAM che si ricorda avere Compute Capability 7.5 e dimensione massima del blocco pari a 1.024. Il profiling di entrambe le soluzioni è stato effettuato sia tramite Nvprof di NVIDIA (ad oggi deprecato), che tramite il più recente e supportato Nsys di NVIDIA.

5.1 Parallel k-means

L'algoritmo converge in 94 iterazioni, sia per il caso sequenziale che per quello parallelo. Questo è necessario per poter effettuare un confronto alla pari delle due soluzioni. Si ricorda che l'algoritmo termina se le modifiche dei centroidi tra un'iterazione e la successiva è minore di una soglia pari a 10^{-10} .

I kernel sono stati eseguiti su griglie di dimensione complessiva 32 e su blocchi di dimensione 1.024, per un totale di 32.768 thread. Per ognuno, sono stati allocati 44 registri in media e non è stata utilizzata la shared memory.

I test effettuati hanno riportato un tempo di esecuzione dell'algoritmo sequenziale di 6,17 secondi e di 1,27 secondi per quello parallelo, con uno speedup pari a 4,84. Sebbene questo non sia elevato, i risultati sono perfettamente in linea con quanto riportato dagli autori del paper originale [1]. Questo è dovuto al fatto che per dataset di dimensioni contenute, nonostante l'uso di una GPU, lo speedup non è significativo.

Il report prodotto da Nvprof è stato riportato nelle tabelle 1 e 2. La maggior parte del tempo di esecuzione delle attività GPU è stato occupato da due kernel: `d_update_centroids` con il 61,66% del totale e `d_assign_cluster` con il 37,45% del totale. Le attività di copia dei dati tra CPU e GPU tramite `cudaMemcpy` rappresentano, invece, una parte minima del tempo. Per quanto concerne le chiamate ad API di CUDA, la quasi totalità del tempo è stata occupata da `cudaDeviceSynchronize` con il 61,65% del totale e da `cudaMalloc` con il 35,08% del totale.

Time(%)	Time	Calls	Avg	Min	Max	Name
61.66%	336.68ms	94	3.5817ms	2.6090ms	8.8210ms	d_update_centroids
37.45%	204.48ms	95	2.1524ms	1.5344ms	4.1704ms	d_assign_cluster
0.63%	3.4613ms	191	18.122us	1.0870us	70.462us	CUDA memcpy HtoD
0.21%	1.1658ms	189	6.1680us	1.2150us	12.127us	CUDA memcpy DtoH
0.05%	262.82us	94	2.7950us	2.2400us	4.7360us	CUDA memcpy DtoD
0.00%	4.9270us	1	4.9270us	4.9270us	4.9270us	CUDA memset

Tabella 1: Risultati delle attività GPU di Parallel k-means

Time(%)	Time	Calls	Avg	Min	Max	Name
61.65%	545.37ms	189	2.8856ms	1.5634ms	8.8239ms	cudaDeviceSynchronize
35.08%	310.32ms	98	3.1665ms	3.6200us	309.64ms	cudaMalloc
1.68%	14.886ms	474	31.405us	4.0070us	327.19us	cudaMemcpy
1.31%	11.544ms	189	61.081us	5.5080us	9.8095ms	cudaLaunchKernel
0.13%	1.1627ms	97	11.986us	6.7320us	261.56us	cudaFree
0.12%	1.0932ms	1	1.0932ms	1.0932ms	1.0932ms	cuDeviceGetPCIBusId
0.02%	133.91us	101	1.3250us	140ns	52.215us	cuDeviceGetAttribute
0.01%	78.607us	1	78.607us	78.607us	78.607us	cudaMemset
0.00%	25.772us	1	25.772us	25.772us	25.772us	cuDeviceGetName
0.00%	1.7540us	3	584ns	215ns	1.1980us	cuDeviceGetCount
0.00%	1.0420us	2	521ns	263ns	779ns	cuDeviceGet
0.00%	464ns	1	464ns	464ns	464ns	cuModuleGetLoadingMode
0.00%	427ns	1	427ns	427ns	427ns	cuDeviceTotalMem
0.00%	277ns	1	277ns	277ns	277ns	cuDeviceGetUuid

Tabella 2: Risultati delle chiamate API CUDA di Parallel k-means

Per quanto concerne l’occupancy, invece, si fa riferimento al report prodotto da Nsys. Si è ottenuto un valore reale pari a 97,67%, raggiungendo un numero di warp attivi per SM pari a 31,75. Sebbene i risultati siano leggermente diversi da quelli teorici massimi, ci riteniamo soddisfatti di quanto ottenuto e attribuiamo queste differenze a possibili overhead legati alle operazioni.

5.2 G-DBSCAN

I kernel sono stati eseguiti su griglie di dimensione complessiva 1.024.144 e su blocchi di dimensione 1.024, per un totale di 1.048.723.456 thread. Per ognuno, sono stati allocati in media 34 registri.

In questo caso, si sono utilizzati sia dei blocchi monodimensionali che bidimensionali come per il kernel `calculate_degs`. Comunque, il numero di thread per blocco è quello riportato.

Unicamente per il kernel `naive_scan`, si è fatto uso della shared memory come discusso in precedenza. La dimensione di questa è tale da memorizzare un numero di interi pari a due volte la dimensione del blocco, quindi $1.024 \times 2 = 2.048$ interi.

L’analisi dei tempi di esecuzione è stata divisa due:

- Creazione del grafo: l’algoritmo sequenziale ha impiegato 117,71 secondi e quello parallelo 4,18 secondi, con uno speedup pari a 28,13. Anche in questo caso, i risultati sono perfettamente in linea con quanto riportato dagli autori del paper originale [3];
- Esplorazione del grafo: l’algoritmo sequenziale ha impiegato 0,04 secondi e quello parallelo 0,12 secondi, con uno speedup di 0,38. Sebbene l’esplorazione GPU sia paradossalmente più lenta, anche questo risultato è coerente con l’analisi effettuata dagli autori. Attribuiamo questo rallentamento alla dimensione ridotta del grafo.

In totale, l’algoritmo sequenziale ha impiegato 117,79 secondi e quello parallelo 4,76 secondi con uno speedup complessivo di 24,73.

Il report prodotto da Nvprof è stato riportato nelle tabelle 3 e 4. La maggior parte del tempo di esecuzione delle attività GPU è stato occupato da due kernel: `make_adj_list` con il 52,74% del totale e `calculate_degs` con il 45,88% del totale. Le attività di copia dei dati tra CPU e GPU

tramite `cudaMemcpy` e i restanti kernel rappresentano, invece, una parte minima del tempo. Per quanto concerne le chiamate ad API di CUDA, la maggior parte del tempo è stata occupata da `cudaDeviceSynchronize` con il 92,03% del totale.

Time(%)	Time	Calls	Avg	Min	Max	Name
52.74%	2.13092s	1	2.13092s	2.13092s	2.13092s	make_adj_list
45.88%	1.85394s	1	1.85394s	1.85394s	1.85394s	calculate_degs
0.88%	35.489ms	123	288.53us	1.0880us	2.6757ms	CUDA memcpy HtoD
0.31%	12.445ms	691	18.010us	1.1840us	4.5783ms	CUDA memcpy DtoH
0.19%	7.7303ms	630	12.270us	2.7840us	151.52us	bfs_gpu_kernel
0.00%	74.433us	24	3.1010us	2.9120us	3.2960us	assign_cluster
0.00%	35.679us	5	7.1350us	1.7280us	24.255us	CUDA memset
0.00%	4.2230us	1	4.2230us	4.2230us	4.2230us	naive_scan
0.00%	3.4880us	1	3.4880us	3.4880us	3.4880us	d_classify_objects
0.00%	2.5280us	1	2.5280us	2.5280us	2.5280us	add_block_sums
0.00%	2.5280us	1	2.5280us	2.5280us	2.5280us	prepend_zero

Tabella 3: Risultati delle attività GPU di G-DBSCAN

Time(%)	Time	Calls	Avg	Min	Max	Name
92.03%	3.99502s	683	5.8492ms	1.8710us	2.13095s	cudaDeviceSynchronize
5.76%	249.88ms	128	1.9522ms	4.7100us	243.87ms	cudaMalloc
2.04%	88.728ms	814	109.00us	8.7840us	5.9731ms	cudaMemcpy
0.14%	6.0824ms	660	9.2150us	6.3650us	43.666us	cudaLaunchKernel
0.03%	1.1904ms	79	15.068us	4.6000us	197.11us	cudaFree
0.00%	126.37us	101	1.2510us	142ns	51.844us	cuDeviceGetAttribute
0.00%	107.64us	5	21.528us	10.469us	34.258us	cudaMemset
0.00%	29.593us	1	29.593us	29.593us	29.593us	cuDeviceGetName
0.00%	6.0520us	1	6.0520us	6.0520us	6.0520us	cuDeviceGetPCIBusId
0.00%	1.7500us	3	583ns	220ns	1.1980us	cuDeviceGetCount
0.00%	1.0060us	2	503ns	310ns	696ns	cuDeviceGet
0.00%	417ns	1	417ns	417ns	417ns	cuModuleGetLoadingMode
0.00%	405ns	1	405ns	405ns	405ns	cuDeviceTotalMem
0.00%	233ns	1	233ns	233ns	233ns	cuDeviceGetUuid

Tabella 4: Risultati delle chiamate API CUDA di G-DBSCAN

Per quanto concerne l’occupancy, invece, si fa riferimento al report prodotto da Nsys. Si è ottenuto un valore reale pari a 98,31%, raggiungendo un numero di warp attivi per SM pari a 31,46. Sebbene i risultati siano leggermente diversi da quelli teorici massimi, ci riteniamo soddisfatti di quanto ottenuto e attribuiamo queste differenze a possibili overhead legati alle operazioni.

6 Valutazione degli stay point

Gli algoritmi hanno prodotto dei risultati differenti ma interessanti. La figura 3 mostra i risultati ottenuti con *k*-means. Come è semplice notare, la soluzione non è in alcun modo adatta al calcolo degli stay point. Questo, però, deriva da caratteristiche intrinseche dell’algoritmo, il quale non riesce a distinguere automaticamente tra punti di interesse e punti di rumore. Inoltre,

non conoscendo a priori il numero dei punti di sosta, si rende ulteriormente meno agevole la clusterizzazione. Per la figura mostrata, si è utilizzato un $k = 23$; questa informazione è stata ottenuta a partire dal numero di cluster rilevati eseguendo G-DBSCAN ed è stata mantenuta uguale per facilitare il confronto.

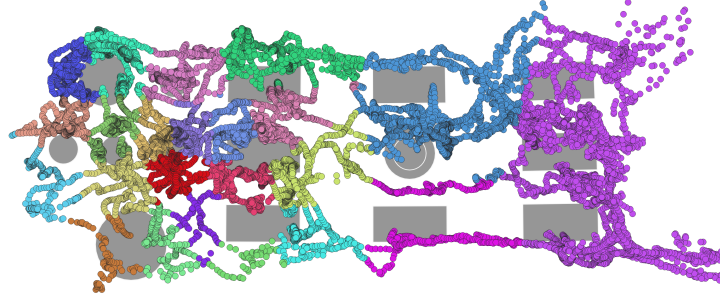
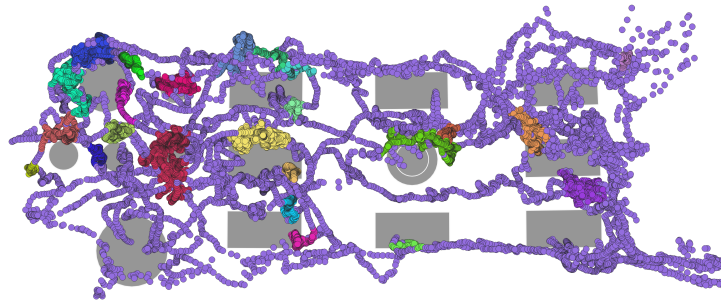
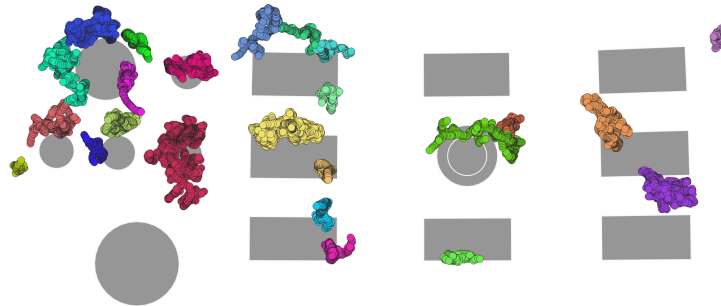


Figura 3: Staypoint rilevati con Parallel k-means

La figura 4, invece, mostra i risultati ottenuti con DBSCAN. La qualità della clusterizzazione è nettamente maggiore e, oltre ai punti di sosta, si possono anche distinguere i punti di rumore come mostrato in 4a. Per osservare unicamente gli stay point, si propone la visualizzazione 4b. I parametri utilizzati per ottenere questi risultati sono $R = 0,0925$ e $min_pts = 50$.



(a) Con i punti di rumore



(b) Senza i punti di rumore

Figura 4: Staypoint rilevati con G-DBSCAN

A seguito delle osservazioni proposte, è possibile stabilire che la soluzione con DBSCAN è nettamente migliore, soprattutto perché si possono distinguere i punti appartenenti ai cluster dai punti di rumore. Questo è fondamentale nella stay point detection perché consente di ottenere risultati più precisi. La classificazione in punto di rumore è strettamente legata ai due parametri R e min_pts citati in precedenza.

7 Conclusione

In questo progetto, abbiamo esaminato due approcci di clustering, l'algoritmo k -means e DBSCAN, nell'ambito dell'analisi dei dati di mobilità e della stay point detection. L'obiettivo posto è implementare questi algoritmi parallelamente per rilevare i punti di permanenza all'interno di traiettorie dei visitatori di un museo. I risultati della valutazione mettono in luce importanti differenze tra le due soluzioni.

Per quanto riguarda l'algoritmo k -means, si sono analizzate sia la versione sequenziale che quella parallela. Il confronto mostra un modesto speedup di circa 4,84, da questo è possibile dedurre che il beneficio derivante dall'elaborazione parallela è limitato dalla dimensione ridotta del dataset. L'occupancy dell'algoritmo è risultata essere del 97,67%, con un numero di warp attivi per SM pari a 31,75.

D'altra parte, DBSCAN ha dimostrato una notevole capacità nella rilevazione dei punti di permanenza. La versione parallela ha mostrato uno speedup notevole di circa 24,73 nella creazione del grafo e 0,38 nella sua esplorazione. L'occupancy dell'algoritmo è stata pari al 98,31%, con un numero di warp attivi per SM pari a 31,46.

Inoltre, la valutazione ha rivelato che DBSCAN offre una migliore capacità di distinzione tra punti di interesse e punti di rumore rispetto a k -means.

In conclusione, le soluzioni parallele implementate si sono dimostrate efficaci, ottenendo risultati sufficienti in termini di speedup e occupancy. L'utilizzo della GPU si è però mostrato non sempre efficiente, soprattutto nel caso dell'esplorazione del grafo; questo dipende strettamente dalla natura del problema in esame.

Riferimenti bibliografici

- [1] Weizhong Zhao, Huifang Ma e Qin He. «Parallel K-Means Clustering Based on MapReduce». In: *Cloud Computing*. A cura di Martin Gilje Jaatun, Gansen Zhao e Chunming Rong. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 674–679.
- [2] David Arthur e Sergei Vassilvitskii. «K-Means++: The Advantages of Careful Seeding». In: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '07. New Orleans, Louisiana: Society for Industrial e Applied Mathematics, 2007, pp. 1027–1035.
- [3] Guilherme Andrade et al. «G-DBSCAN: A GPU Accelerated Algorithm for Density-based Clustering». In: *Procedia Computer Science* 18 (2013). 2013 International Conference on Computational Science, pp. 369–378.